Tree-budget search: parameter corollaries and a paper proof of bit-cost bounds

ChatGPT-6 Astra

5 November 2026

Abstract

We specify a binary-encoded matrix decision and explicit-witness task, and give a paper construction of an explicit-vector implementation of the existing ordered, association-list memoized search. Including candidate enumeration, legality tests, both lookup and insertion key comparisons, arithmetic, and witness construction, its bit-cost RAM time is $O(S^3(M+1)(\ell+1)^5)$. Here $S=\prod_{v\neq o}(H_v+1)$, $M=\sum_{v\neq o}H_v$, and ℓ is the total input encoding length. This gives paper-level FPT upper bounds for M and for the pair (t,U). The representation simulation and the bit-cost theorem in this note are not Lean theorems and are not a time bound for the current compiled Lean program. Existing Lean results certify the search semantics and logical operation counts used as premises.

1 Status and scope

The source baseline is the research commit `1f4774c`. Its modules `TreeBudgetSearchStatsBounds`, `TreeBudgetCandidateCount`, `TreeBudgetSearchCostBounds`, and `TreeBudgetStateParameters` supply the logical bounds below. The accompanying `TreeBudgetParameterizedBounds` module closes the logical parameter corollaries; its exact build and audit status is recorded separately in the repository status document.

Sections 3–6 are mathematical proofs about an explicit representation specified here, not claims of a completed Lean refinement or compiler-cost proof. No performance measurements are used. In particular, an arbitrary value of type `Fin n -> Nat` is not assumed to have constant-time evaluation. The task starts from multiplicities H; converting a different demand encoding into H, expanding rows into physical tours, and compressing certificates are outside this task.

2 Logical parameter corollaries

Let $V=\{0,\dots,n-1\}$, where $n\geq 1$, be the vertices of a rooted tree with root o and parent map par. Set

$$S=\prod_{v\neq o}(H_v+1),\quad M=\sum_{v\neq o}H_v,\quad t=|\{v\neq o: H_v>0\}|,\quad U=\max(\{H_v: v\neq o\}\cup\{0\}).$$

Write $k_0=\mathrm{rc}(H)=\sum_{v\neq o,\,\mathrm{par}(v)=o}H_v$. For the formal cold entry, let P,E,Q,C,I,L denote the candidate-list length, expansions, candidate checks, state calls, insertion scans, and lookup key comparisons, respectively. The established bounds are

$$P,E\leq S,\quad Q\leq S^2,\quad C\leq 1+S^2,\quad I\leq\frac{S(S-1)}{2},\quad L\leq S+S^3. \tag{1}$$

The final cache has exactly E entries. These initial-entry bounds do not apply to an arbitrary hot cache or arbitrary candidate list.

Lemma 2.1. *We have $k_0 \le M \le tU$, $S \le 2^M$, and $S \le (U+1)^t$. A successful cold-entry row witness has exactly k_0 rows, hence at most M rows.*

Proof. The root children form a subset of the nonroot vertices, and all summands are nonnegative, giving $k_0 \le M$. Delete zero summands from M and bound each of the remaining t summands by U. Induction gives $h+1 \le 2^h$ for every $h \in \mathbb{N}$: the successor step uses $h+2 \le 2(h+1)$. Multiplication over nonroot vertices gives $S \le 2^{\sum H_v}$. Removing unit factors from S leaves t factors, each at most $U+1$. Finally, the residual-witness specification at remaining count k requires a list of length exactly k. Apply soundness at $k = k_0$. Empty products equal one, so the argument also covers $M = t = U = k_0 = 0$. □

Corollary 2.2. *For either $A = 2^M$ or $A = (U+1)^t$,*

$$P, E \le A, \quad Q \le A^2, \quad C \le 1 + A^2, \quad I \le \frac{A(A-1)}{2}, \quad L \le A + A^3.$$

Proof. Substitute $S \le A$ in (1). For the insertion bound, use monotonicity of $\sum_{j=0}^{a-1} j = a(a-1)/2$ on natural numbers. This avoids treating truncated subtraction as an independently monotone expression. These are logical-count conclusions, without a time claim. □

3 A fixed encoded task

Definition 3.1 (Input and output)**.** The input consists of an explicitly listed parent array on V, a root index o, a list of nonnegative binary integers H_v, signed rational service fees $s_v = a_v/b_v$, and a rational budget $T = a_T/b_T$, where all denominators are positive integers. Fractions need not be reduced. Let $\ell \ge n$ be the total length of this encoding, including delimiters and signs. The parent array is required to describe a rooted tree: $\mathrm{par}(o) = o$, and every other vertex reaches o by following parents. Malformed encodings may be rejected before search.

Replace H_o by zero after reading the input, and call the normalized vector H. The task is to decide whether there exists a list of $k_0 = \mathrm{rc}(H)$ rows $p \in \mathbb{N}^n$, each satisfying

$$p_o = 0, \qquad 0 \le p_v \le H_v \quad (v \ne o), \qquad \mathrm{rc}(p) = 1, \tag{2}$$

$$v \ne o,\ p_v > 0,\ \mathrm{par}(v) \ne o \implies p_{\mathrm{par}(v)} > 0, \tag{3}$$

$$\sum_{v \ne o} \big(s_v + \mathbf{1}_{\mathrm{par}(v) \ne o} s_{\mathrm{par}(v)}\big) p_v \le T, \tag{4}$$

whose nonroot column sums are H_v. On success the witness task outputs that explicit row matrix in binary; on failure it outputs NO. The decision task may run the same algorithm and discard the witness.

These conditions are exactly `RowValid` and the initial matrix condition, using `cost_eq` for the linear service cost. Condition (3) is a local parent test, not an enumeration of ancestor paths or subsets. No sign restriction on s_v or T is needed. An optional binary row-count input m can be handled by rejecting $m \ne k_0$: summing $\mathrm{rc}(p) = 1$ over rows and using the column sums forces $m = k_0$. This note does not analyze the separate demand-to-flow wrapper or root-branch decomposition.

Remark 3.2 (Why normalization matters)*.* The low-level logical count theorems allow an arbitrary root coordinate. Matrix feasibility, however, ignores that coordinate whereas the residual base test checks $r = 0$ at every coordinate. The mathematical entry here uses $H_o = 0$, exactly as `searchMemo` normalizes its input. Without this step, a nonzero root coordinate could incorrectly force failure for the intended matrix task. The values S, M, t, U, k_0 are unchanged by normalization.

4 Representation and semantic correspondence

Use materialized length-n binary vectors, not unevaluated function closures. Enumerate all vectors with $p_o = 0$ and $0 \le p_v \le H_v$ in the coordinate order of `boundedVectors`, then stably filter by (2)–(4). Store the resulting ordered list once, and reuse that same list in every recursive call.

The cache is an ordered association list of keys (k, r) and answers. An answer is either failure or a successful row list; a cache miss is a third, distinct outcome. In particular, a cached failure is a hit and a successful empty list is not a failure. Use these operations:

1. Search the cache from its head, comparing full keys, and return the first matching answer if present.

2. On a miss with $k = 0$, return success with the empty list exactly when every coordinate of r is zero.

3. On a miss with $k > 0$, scan the full candidate list in order. For each $p \le r$, materialize $r - p$ and call the solver with $k - 1$. A failing child passes its updated cache to the next candidate. A successful child returns p prepended to its rows immediately, without visiting the unused suffix.

4. After the miss has been evaluated, filter the entire returned cache to remove matching keys, then prepend the new key and answer. The filter is still executed when the parent key is known to be absent.

Start with $(k, r) = (k_0, H)$ and the empty cache. Immutable vectors and row lists may be shared by references, or copied; the conservative bound below allows copying an entire answer whenever an answer is transferred.

Lemma 4.1 (Paper simulation)**.** *Interpret a stored vector x as the function $v \mapsto x_v$. Under this interpretation, the algorithm just specified returns the same answer and ordered cache as `memoRows`, and its logical events C, E, Q, L, I coincide with `memoRowsStat`.*

Proof. First induct on the number of coordinates. The nested coordinate loops produce precisely the list obtained by the range, flat-map and cons steps of `boundedVectors`, in the same order. Array tests for (2)–(4) express exactly `RowValid`; stable filtering therefore gives the identical candidate list.

Next, array equality is extensional function equality on the finite domain. Coordinatewise guarded subtraction agrees with natural-number subtraction, and the all-zero array test agrees with function equality to zero. Induct on k, with a subsidiary induction on the scanned suffix. Lookup visits identical entries and gives the same first hit or miss. At zero depth both programs make the same zero test. At positive depth, a rejected guard moves to the same suffix; an accepted guard invokes the induction hypothesis on the same child state and input cache. On failure the suffix induction starts with the child's returned cache. On success both prepend the same row and stop. Finally the full-key filter and prepending operation agree entry by entry.

Counting a state call, a miss, a candidate test, each visited lookup entry, and each visited insertion-filter entry in these matching branches gives the stated equality of events. This proves a mathematical simulation; the explicit-vector implementation and this refinement have not been encoded or checked in Lean in this package. □

5 Bits, arithmetic, and the complete cost ledger

Use a bit-cost RAM with binary storage and exact integer arithmetic. Reading or writing a w-bit value or address is charged $O(w)$, not unit cost for unbounded words. Schoolbook

addition, subtraction, comparison, multiplication, and long division give $O(w^2)$ upper bounds for operands of at most w bits. Arrays are addressed by binary indices; their address costs are included. Let

$$B = \ell + \lceil \log_2(M+2) \rceil + 2.$$

All constants in the following bounds are absolute and independent of the parameters. We deliberately use loose polynomial bounds.

Lemma 5.1 (Value and rational size)**.** *Every residual coordinate is between zero and H_v, and every reachable remaining count is at most $k_0 \le M$. All arithmetic needed for candidate legality can be performed with $O(B)$-bit integers after polynomial-time preprocessing.*

Proof. The residual assertion follows by induction on guarded subtraction; k only decreases. Nonroot coordinates are at most M. To avoid any repeated-denominator-growth assumption, set

$$D = b_T \prod_{v \in V} b_v > 0, \quad A_v = a_v(D/b_v), \quad A_T = a_T(D/b_T), \quad c_v = A_v + \mathbf{1}_{\mathrm{par}(v) \neq o} A_{\mathrm{par}(v)}.$$

The quotients are exact integers. The sum of the bit lengths of all input denominators is at most ℓ, so D has $O(\ell)$ bits. The integers A_v, A_T, c_v have $O(\ell)$ bits as well. Multiplication by the positive D changes (4) into exactly $\sum_{v \neq o} c_v p_v \le A_T$. For every partial sum, its absolute value is at most $\max_v |c_v|\, M$, hence its length is $O(B)$. This also handles negative fees, cancellations, and $M = 0$. Forming the products and exact quotients uses $O((n+1)B^2)$ bit work under schoolbook algorithms. There is no rational arithmetic in the search after candidates have been filtered. □

Lemma 5.2 (Representation and address size)**.** *An implementation as above can use addresses of $O(B)$ bits throughout, even if discarded allocations are never reclaimed. A key requires $O((n+1)B)$ bits, and any successful answer requires $O((M+1)(n+1)B)$ bits when copied explicitly.*

Proof. The key claim follows from Lemma 5.1. Soundness of cached answers and $k \le k_0$ bound their list lengths by M; each row has n coordinates of $O(B)$ bits. Candidate storage has $P \le S$ rows. Along the depth-first evaluation there are at most $M+1$ state frames; scans and cache traversals may instead be implemented by loops.

A finite implementation uses at most a fixed polynomial in $S, M+1, n+1, B$ elementary allocations: enumerate at most $O((n+1)S)$ partial vectors, perform at most $C + Q + L + I + E + 1 = O(S^3)$ search events, and permit a copy of at most M rows, a key, and constant bookkeeping per event. Rebuilding filtered cache lists costs one such copy per insertion scan. Even allocating a fresh binary cell for every arithmetic temporary multiplies this quantity only by a polynomial in B. First count abstract records and scalar slots, treating each reference as one slot independently of its eventual bit width. Give these slots consecutive integer identifiers. Allocation indices therefore have length $O(\log S + \log(M+2) + \log(n+1) + \log(B+1))$. But $\log_2 S = \sum_{v \neq o} \log_2(H_v + 1) = O(\ell)$, and $n \le \ell$, so this is $O(B)$. Encoding each reference with that width multiplies bit storage by only $O(B)$, whose logarithm is still $O(B)$. This argument bounds the number of allocations before charging their address lengths; it does not assume the time bound it will be used to prove. □

Lemma 5.3 (Cost ledger)**.** *The following conservative bounds include bit arithmetic and addressing:*

Work	*Bit-cost upper bound*
Parse and validate the explicit parent tree; normalize; compute k_0; clear rational denominators	$O((n+1)^2B^3)$
Enumerate all bounded vectors and test legality	$O(S(n+1)^2B^3)$
Lookup and insertion full-key comparisons, including list reconstruction and possible copied answers	$O((L+I)(M+1)(n+1)^2B^3)$
Guards, residual subtraction, zero tests, calls, expansion and answer handling	$O((Q+C+E+1)(M+1)(n+1)^2B^3)$
Serialize an explicit successful matrix	$O((M+1)(n+1)^2B^3)$

Proof. Following parent pointers for at most n steps from every vertex checks the rooted-tree condition in polynomial bit time, with rejection on an invalid index or a cycle. Finite summation computes k_0, and Lemma 5.1 handles rational preprocessing.

The product of coordinate range sizes is exactly S; fixing the root to zero contributes one. In nested-loop enumeration, each prefix level contains at most S nodes, since all remaining range sizes are at least one. There are at most $(n+1)S$ nodes in total, including in the all-zero case. Incrementing counters and materializing vectors costs polynomial work in n, B per output. Each legality test checks bounds, a sum over root children, and at most one parent condition per vertex. The scaled cost uses at most n products and additions on $O(B)$-bit integers. These give the stated (loose) enumeration and filtering bound.

Every full-key equality inspects the count and at most n residual coordinates. It costs $O((n+1)B^2)$ including accesses, with early termination only reducing the cost. Both lookup and insertion filtering perform this operation: their combined number is $L+I$, not L. Insertion also traverses and rebuilds the filtered list. Charge each visited/rebuilt entry to its insertion scan; charge the prepended entry to the expansion. Even copying the stored answer per rebuilt entry is covered by the $M+1$ factor.

There is one coordinatewise guard per candidate check, at most one materialized subtraction per accepted guard, and at most one all-zero test per zero-depth expansion. Branch overhead, decrementing k, and empty-list tests are charged to calls and scans. A hit may copy an answer of at most M rows, and a successful child may build one new row list; allowing a full answer copy per such event is still within the displayed bound. Lemma 5.2 accounts for address lengths. Finally output has at most M rows with n binary coordinates. Neither physical-tour expansion nor optional certificate compression is performed. All phases of the specified task have now been charged. □

6 The resulting paper time theorem

Theorem 6.1 (Explicit-vector bit-cost implementation)**.** *For the task in Definition 3.1, there exists a deterministic explicit-vector implementation of the ordered memoized search, with the semantic correspondence of Lemma 4.1, whose time satisfies*

$$\text{Time} = O(S^3(M+1)(n+1)^2B^3) = O(S^3(M+1)(\ell+1)^5)$$

in the specified bit-cost RAM model, for decision and explicit matrix output. This is a paper theorem for that implementation, not a newly machine-checked time theorem for the Lean executable.

Proof. Sum Lemma 5.3 and apply (1). Since $S \geq 1$,

$$1 + S + C + Q + E + L + I = O(S^3).$$

For example $I \leq S^2/2$, while the only cubic term is the lookup bound. This gives the first estimate, including preprocessing and output. Each input coordinate has at most ℓ bits and

$n \le \ell$, so $M < n2^\ell$ when $M > 0$, and $\log_2(M+2) = O(\ell+1)$ also when $M = 0$. Thus $B = O(\ell+1)$, proving the second estimate. □

Corollary 6.2 (Parameter classifications at the paper level)**.** *The encoded decision and explicit matrix-witness tasks admit bit-cost RAM upper bounds*

$$O\big(2^{3M}(M+1)(\ell+1)^5\big) = 2^{O(M)}(\ell+1)^{O(1)}$$

and

$$O\big((U+1)^{3t}(tU+1)(\ell+1)^5\big).$$

Consequently these tasks are FPT for M and for the joint parameter (t, U), for the paper implementation and representation specified here.

Proof. Use Lemma 2.1 in Theorem 6.1. The exponent of $\ell+1$ is the fixed constant five. All other factors depend only on the indicated parameter. In particular, $M+1 \le 2^M$ for natural M, including zero. No XP intermediate result is needed. □

Remark 6.3 (Other standard models)*.* These parameter classifications do not rely on free unbounded addressing. For example, a deliberately naive multitape Turing-machine simulation has polynomial overhead. If the charged RAM time is $F \ge 2$, at most F memory operations occur and no written value or address has more than $O(F)$ bits. Store the touched cells as a sequential list of address-value records, of total size $O(F^2)$. Scanning it for each operation gives $O(F^3)$ time, with the already charged arithmetic performed on tapes. This coarse simulation suffices for $2^{O(M)}\ell^{O(1)}$ and $\mathrm{FPT}(t, U)$ on a standard tape model as well; it does not preserve the sharper cubic power of S in Theorem 6.1.

Remark 6.4 (Limits of the conclusion)*.* No binary-input polynomial bound or XP bound for t alone follows. On the one-edge tree, let $H_v = U$, $s = 0$, and $T = 1$. Every legal row has $p_v = 1$, so the explicit output has $U = k_0$ rows although $t = 1$ and the input length is $O(\log(U+2))$. This output task already requires $\Omega(U)$ output symbols. It says nothing against a faster decision algorithm or a directly constructed compressed certificate. If the multiplicities are instead given in unary, $M, U \le \ell$; the displayed bound then gives an XP form in t. None of these statements yields a classification by tree height, treewidth, or vehicle count alone, or a hardness reduction. Nor is $2^{O(M)}$ a polynomial in the numeric value M.

7 What remains to formalize

The paper proof fixes the computational task and discharges its mathematical cost ledger. A future Lean package should define the finite-vector and integer-scaled legality representation, prove its simulation of the ordered search, attach a bit-cost semantics to its primitive operations, and then derive Theorem 6.1. The existing function-valued Lean definitions have not been replaced in this package. The current verified contribution is still the logical-count and parameter layer; the additional time and FPT statements above have paper-proof status only.